

YaPcore whitepaper — plain English edition

YapLabs · companion to the technical whitepaper

Version **0.3** · September 2026

Pairs with: [YAPCORE_WHITEPAPER.md](#) (YAP-WP-16T-001)

This is the same story as the technical whitepaper, written for people who don't live in systems engineering.

If you want the academic / engineer version (full plugin catalog, MMO, data plane, smokes), use the [technical whitepaper](#).

The big idea (abstract)

Most Minecraft-style servers put almost everything important on **one main worker**: updating the world, talking to plugins, and a lot of network-related work. That keeps the design simple — until the server gets busy, and then that one worker becomes a traffic jam.

YaPcore splits the product into three clear pieces:

1. **YaP-Folia** — our multi-threaded game engine (a fork of PaperMC Folia, not the stock download) runs the world.
2. **YaPEngine** — a slim “chassis” handles networking, menus, databases, and ops — **not** the world heartbeat.
3. **YaP Link** — our network front door when you run more than one game box.

Ordering rules keep things from happening out of turn. Older-style plugins still work through a careful compatibility path when needed. Java and Bedrock players can join toward the same world story. New plugins are guided into lanes so heavy database work doesn't freeze the fun parts of the game.

On top of that stack we **ship** the plugins most networks used to assemble themselves: ranks, chat, moderation, player data / economy / shops / auction house, protect, world tools, regions, map, factions — plus an optional gameplay pack (vehicles, stacker, and a full skills/combat/crafting MMO).

1. What's the problem?

The everyday version

Imagine a store with **one cashier** for ringing up customers, restocking shelves, answering the phone, and updating the inventory computer. When it's quiet, that works. When it's busy, the line piles up.

That's roughly how classic Minecraft servers work: one "main thread" does the authoritative world updates. If a plugin spends that time waiting on a database, the whole world feels laggy. If you let lots of workers change the world at once **with no rules**, you get broken chunks and inventory glitches.

Operators also used to install **ten separate community plugins** (permissions, essentials, CoreProtect, WorldGuard, Velocity, Geyser, ...) and hope they all got along on Folia.

Bedrock players also use a different connection style (UDP) than Java (TCP), but operators usually still want **one shared world**.

What YaPcore is trying to contribute

1. **Three layers — YaP-Folia** runs the game; **YapEngine** runs edge/I/O; **YaP Link** fronts multi-backend networks.
2. **Order tickets (SequenceToken)** — work can move between chassis workers without everything sharing one giant lock.
3. **A Compatibility Bridge** — older plugins can still change the world safely by waiting for the right window.
4. **Java + Bedrock on one product story** — both can join; optionally even on the same port number.
5. **Three ways to extend the server** — Folia-aware plugins, YaP plugins, and smaller "fine-tune" modules — all drop into one `plugins/` folder.
6. **Shipped YaP plugins** — CORE+NETWORK by default; GAMEPLAY + MMO opt-in — so you are not piecing together LuckPerms + EssentialsX + ... for a normal survival/network.

What we are *not* promising

We don't claim "every Paper plugin works on YaP-Folia day one" (same reality as Folia), and we're not shipping stock Folia as the product jar. We grow compatibility based on what people actually need. Bedrock join and play-depth smoke are green; Wave 2 documents Limited/Out rows (Floodgate-only forms, anvil/smithing) instead of silent Partial.

Where the product is today (September 2026)

We use **YaP-Folia** for the real Minecraft game by default (`folia-jar-source=build`).

YapEngine runs the **slim edge/I/O chassis** around it (not world tick). **YaP Link** is our Velocity-class front door (**phases 0-6 shipped**). Legacy **Paper + Phase 3** spatial tick is **done as code** but **off by default**. CORE+NETWORK plugins ship by default; GAMEPLAY/MMO is opt-in. Playerdata **shops + auction house** are **on** by default (jobs stay

off when skills are used). The product is aimed at **busy / high-pop** servers. **Citeable** population MSPT uses **fullcite** (100 active bots + fixtures); yapcore **-12.4%** vs stock Folia with ship knobs (smart budget + microtick + subregion partition); join verified at 100/200 bots — [YAPCORE_WHITEPAPER.md](#). Phase 4 dual-stack join DoD is green.

Details: [YAPCORE_WHITEPAPER.md](#) · [QUICK_START.md](#) · [YAPCORE_WHITEPAPER.md](#) · [YAP_LINK.md](#) · [technical whitepaper §6-13](#).

2. How this compares to other approaches

- **Paper / Purpur** improve the classic Bukkit model (still one main tick).
- **Upstream Folia** provides region-based multithreading — we **fork** it as **YaP-Folia**.
- **Proxies** (Velocity; YaP Link) route players but don't usually *own* the world.

YaPcore sits as:

Keep **YaP-Folia for the kitchen**, a **slim YapEngine edge/I/O chassis**, and **YaP Link** at the front door for multi-backend networks — with **YaP plugins** already in the pantry.

3. Architecture in plain English

3.1 Three layers + chassis channels

Layer	Plain English
YaP Link	Front door for big networks — sends players to the right backend
YapEngine chassis	Netty edge, dual-stack, plugin bridge, menus, DB/HTTP — not world tick
YaP-Folia	The kitchen — chunks, mobs, redstone, commands on many region workers

Inside the chassis, numbered “channels” (1-16) have fixed jobs: watchdog, traffic shaping, bridge, menus, heavy I/O, telemetry. On the product path, world tick is **not** on those chassis cores — it is on YaP-Folia’s region pool.

3.2 Order tickets

When work moves between workers, it carries a **SequenceToken** so one player’s actions don’t get reordered into nonsense, while different players can still progress in parallel.

3.3 Memory

Production scripts lean on modern garbage collection (**Generational ZGC**) and can pin to NUMA nodes on big machines.

4. Plugins without freezing the world

Think of three lanes:

Lane	Good for	Bad for
SYNC	Changing blocks, inventories, teleports	Waiting on a database
HEAVY	Databases, HTTP, disk	Touching the world directly
UI	Menu polish	Authoritative world writes

YaP first-party plugins declare `folia-supported: true` and schedule through those rules.

5. What you get “in the box”

Always (CORE+NETWORK): shared MariaDB pool, ranks, player sync + economy + shops + AH + claims, moderation, essentials QoL, chat, packs, PlaceholderAPI, pregen, protect, world tools, regions, NPCs, TAB, Discord webhooks, anti-cheat lite, lag guard, web map, factions, Bedrock identity/UI bridge.

Optional (GAMEPLAY): vehicles, mob stacker, gameplay knobs, and the MMO pack (13 skills, custom combat, crafting, quests/bosses, abilities, guilds, minigames, Bedrock MMO UI).

Full tables: [technical whitepaper §6](#).

6. Networking in plain English

- **Java players** connect over TCP; version translation lives in our chassis (no separate Via jar).
 - **Bedrock players** connect over UDP through our own crossplay path (no Geyser jar).
 - **YaP Link** can sit in front when you run multiple game servers.
 - Resource packs can be served over HTTP; ops use a browser dashboard on port **8080**.
-

7. How we check that it works

Unit tests cover plugin and API behavior. Operators validate with a local boot after `./scripts/seed-defaults.sh`.

8. Honest caveats

- Paper-only plugins are not a free pass on Folia.
 - Protocol and Bedrock fidelity: Wave 2 matrix in CROSSPLAY (Floodgate-only forms Limited; anvil/smithing Out).
 - Fancy Folia performance patches stay **off** until load-tested on your hardware.
 - Dashboard tabs for Factions, Guilds, Games, Disasters, Stacker, Protect, and Regions are interactive for ship ops; leftover plugins stay on Plugin settings by design.
 - Pregen is Folia-safe (region-scheduled loads + chunk tickets).
-

9. Bottom line

YaP-Folia cooks the world on many threads. **YapEngine** runs the front-of-house (network, menus, databases). **YaP Link** seats guests across multiple dining rooms. **YaP plugins** stock the shelves so operators are not assembling a grocery list of community jars for a normal network.

For the full engineering write-up, tables, and status matrix, read [YAPCORE_WHITEPAPER.md](#).

Where to go next

If you are...	Start here
Non-tech / just curious	YAPCORE_WHITEPAPER_PLAIN_ENGLISH.md
Reading the engineer paper	YAPCORE_WHITEPAPER.md
Running a server	QUICK_START , WEB_DASHBOARD
Writing plugins	PLUGINS , VEHICLES
Working on the fork	QUICK_START , YAPCORE_WHITEPAPER.md

Citation (technical paper)

```
@techreport{yapcore2026sixteen,  
  title      = {YaPcore: YaP-Folia Game Authority, Slim Edge Chassis, Native Network Stack,  
  ...
```

```
and First-Party Plugin Suite},  
  author      = {{YapLabs}},  
  institution = {YapLabs},  
  year        = {2026},  
  month       = sep,  
  number      = {YAP-WP-16T-001},  
  note        = {Technical whitepaper, YaPcore 0.3}  
}
```

License

YaPcore is free software under the **GNU GPLv3** — [LICENSING.md](#).